

Taller de Investigacion: Deteccion y Eliminacion de RFI con Deep Learning

Prof. Miguel Cárcamo

25 de septiembre de 2025

Índice

1. Introduccion	3
2. Astrofisica Basica Aplicada a Radioastronomia	3
2.1. Fundamentos de Radioastronomia	3
2.2. Ecuacion de Transferencia Radiativa	3
2.3. Temperatura de Brillo	3
2.4. Ejercicio de Investigacion 1	3
3. Radio-Interferometria	4
3.1. Principio de Interferometria	4
3.2. Resolucion Angular	4
3.3. Ejercicio de Programacion 1	4
4. Naturaleza de la RFI	7
4.1. Definicion y Origenes	7
4.2. Caracteristicas de la RFI	7
4.3. Ejercicio de Investigacion 2	7
5. Tipos de Interferencia	7
5.1. RFI de Banda Angosta	7
5.2. RFI de Banda Ancha	8
5.3. RFI Transitoria	8
5.4. Ejercicio de Programacion 2	8
6. Tecnicas de Flagging	9
6.1. AOFlagger	9
6.2. Algoritmo SumThreshold	10
6.3. Umbrales Estadisticos	10
6.4. Ejercicio de Programacion 3	10
7. Repaso Matematico	11
7.1. Transformada de Fourier	11
7.1.1. Matrices de Transformacion	12
7.1.2. Matriz Transpuesta y Adjunta	12

7.2.	Espacios de Visibilidades	12
7.3.	Algebra Lineal y No Lineal	13
7.3.1.	Problemas Lineales	13
7.3.2.	Problemas No Lineales en Sintesis de Imagenes	13
7.3.3.	Ecuacion No Lineal General	13
7.3.4.	Algoritmos No Lineales	14
7.3.5.	Ejercicio de Programacion 4	14
7.3.6.	Ejercicio de Programacion 4.5	16
7.4.	Minimos Cuadrados	17
7.5.	Ejercicio de Programacion 5	17
8.	Ejercicios Integradores	19
8.1.	Ejercicio de Investigacion 3	19
8.2.	Ejercicio de Programacion 6	20
9.	Conclusiones y Proximos Pasos	21
9.1.	Recursos Adicionales	21

1. Introduccion

Este taller aborda los fundamentos teoricos y practicos para la deteccion y eliminacion de interferencia de radiofrecuencia (RFI) utilizando tecnicas de aprendizaje profundo. El contenido se estructura en modulos teoricos seguidos de ejercicios de investigacion y programacion.

2. Astrofisica Basica Aplicada a Radioastronomia

2.1. Fundamentos de Radioastronomia

La radioastronomia estudia objetos celestes mediante la deteccion de ondas de radio. Las fuentes principales incluyen:

1. **Emission termica:** Radiacion de cuerpo negro de objetos calientes
2. **Emission sincrotron:** Electrones relativistas en campos magneticos
3. **Emission de linea:** Transiciones moleculares especificas

2.2. Ecuacion de Transferencia Radiativa

La intensidad especifica I_ν sigue la ecuacion de transferencia:

$$\frac{dI_\nu}{ds} = -\kappa_\nu I_\nu + j_\nu \quad (1)$$

donde κ_ν es el coeficiente de absorcion y j_ν es el coeficiente de emision.

2.3. Temperatura de Brillo

Para objetos extendidos, definimos la temperatura de brillo:

$$T_B = \frac{c^2 I_\nu}{2k_B \nu^2} \quad (2)$$

2.4. Ejercicio de Investigacion 1

Objetivo: Analizar el espectro de temperatura de brillo de diferentes objetos astromicos.

Tareas:

1. Investigar los rangos tipicos de temperatura de brillo para:
 - Nucleos galacticos activos (AGN)
 - Remanentes de supernova
 - Nubes moleculares
2. Comparar con la temperatura de ruido del sistema de un radiotelescopio tipico
3. Calcular la relacion senal-ruido esperada

3. Radio-Interferometria

3.1. Principio de Interferometria

La interferometria aprovecha la correlacion entre senales de multiples antenas. La visibilidad compleja $V(u, v)$ se relaciona con la distribucion de brillo $I(l, m)$ mediante:

$$V(u, v) = \int \int I(l, m) e^{-2\pi i(ul+vm)} dl dm \quad (3)$$

donde:

- $V(u, v)$: visibilidad compleja en coordenadas (u, v)
- $I(l, m)$: distribucion de brillo en el cielo
- (u, v) : coordenadas en el plano de Fourier (longitudes de onda)
- (l, m) : cosenos directores (radianes)

3.2. Resolucion Angular

La resolucion angular maxima esta dada por:

$$\theta_{res} \approx \frac{\lambda}{B_{max}} \quad (4)$$

donde:

- θ_{res} : resolucion angular maxima (radianes)
- λ : longitud de onda de observacion (metros)
- B_{max} : linea base maxima (metros)

3.3. Ejercicio de Programacion 1

Objetivo: Simular configuracion de antenas y transformacion de coordenadas UVW.

Fórmulas necesarias:

1. Transformacion ECEF a ENU:

$$\begin{bmatrix} E \\ N \\ U \end{bmatrix} = \mathbf{R}_{ECEF}^{ENU} \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} \quad (5)$$

donde:

- E, N, U : coordenadas East-North-Up (metros)
- X, Y, Z : coordenadas ECEF (metros)
- $\mathbf{R}_{ECEF}^{ENU}$: matriz de rotacion ECEF a ENU

La matriz de rotacion es:

$$\mathbf{R}_{ECEF}^{ENU} = \begin{bmatrix} -\sin \lambda & \cos \lambda & 0 \\ -\sin \phi \cos \lambda & -\sin \phi \sin \lambda & \cos \phi \\ \cos \phi \cos \lambda & \cos \phi \sin \lambda & \sin \phi \end{bmatrix} \quad (6)$$

donde:

- λ : longitud geografica del observatorio (radianes)
- ϕ : latitud geografica del observatorio (radianes)

2. Transformacion ENU a coordenadas horizontales (Alt-Az):

$$\begin{bmatrix} E_{hor} \\ N_{hor} \\ U_{hor} \end{bmatrix} = \mathbf{R}_{ENU}^{Hor} \begin{bmatrix} E \\ N \\ U \end{bmatrix} \quad (7)$$

donde:

- $E_{hor}, N_{hor}, U_{hor}$: coordenadas en sistema horizontal
- E, N, U : coordenadas ENU (metros)
- $\mathbf{R}_{ENU}^{Hor}$: matriz de rotacion ENU a horizontal

La matriz de rotacion ENU a horizontal es:

$$\mathbf{R}_{ENU}^{Hor} = \begin{bmatrix} \cos Az & -\sin Az & 0 \\ \sin Az & \cos Az & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (8)$$

donde:

- Az : acimut del objeto observado (radianes)

3. Transformacion horizontal a ecuatorial (XYZ):

$$\begin{bmatrix} X_{eq} \\ Y_{eq} \\ Z_{eq} \end{bmatrix} = \mathbf{R}_{Hor}^{Eq} \begin{bmatrix} E_{hor} \\ N_{hor} \\ U_{hor} \end{bmatrix} \quad (9)$$

donde:

- X_{eq}, Y_{eq}, Z_{eq} : coordenadas ecuatoriales
- $\mathbf{R}_{Hor}^{Eq}$: matriz de rotacion horizontal a ecuatorial

La matriz de rotacion horizontal a ecuatorial es:

$$\mathbf{R}_{Hor}^{Eq} = \begin{bmatrix} \cos \delta \cos H & -\sin \delta \cos H & \sin H \\ \cos \delta \sin H & -\sin \delta \sin H & -\cos H \\ \sin \delta & \cos \delta & 0 \end{bmatrix} \quad (10)$$

donde:

- H : angulo horario del objeto observado (radianes)
- δ : declinacion del objeto observado (radianes)

4. Transformacion ecuatorial a UVW:

$$\begin{bmatrix} u \\ v \\ w \end{bmatrix} = \mathbf{R}_{Eq}^{UVW} \begin{bmatrix} X_{eq} \\ Y_{eq} \\ Z_{eq} \end{bmatrix} \quad (11)$$

donde:

- u, v, w : coordenadas UVW (metros)
- $\mathbf{R}_{Eq}^{UVW}$: matriz de rotacion ecuatorial a UVW

La matriz de rotacion ecuatorial a UVW es:

$$\mathbf{R}_{Eq}^{UVW} = \begin{bmatrix} \sin H & \cos H & 0 \\ -\sin \delta \cos H & \sin \delta \sin H & \cos \delta \\ \cos \delta \cos H & -\cos \delta \sin H & \sin \delta \end{bmatrix} \quad (12)$$

5. Conversion a longitudes de onda:

$$\mathbf{b}_{uvw} = \frac{\mathbf{b}_{UVW}}{\lambda} = \frac{\mathbf{b}_{UVW} \cdot f}{c} \quad (13)$$

donde:

- $\mathbf{b}_{uvw}$: vector baseline en coordenadas UVW (longitudes de onda)
- $\mathbf{b}_{UVW}$: vector baseline en coordenadas UVW (metros)
- λ : longitud de onda de observacion (metros)
- f : frecuencia de observacion (Hz)
- c : velocidad de la luz (m/s)

Tareas:

1. Implementar las matrices de rotacion ECEF-ENU, ENU-Horizontal, Horizontal-Ecuatorial y Ecuatorial-UVW
2. Generar posiciones de antenas para configuraciones VLA y ALMA
3. Calcular baselines entre todas las parejas de antenas
4. Aplicar la secuencia completa de transformaciones de coordenadas
5. Calcular cobertura UVW para una observacion de 6 horas
6. Visualizar la cobertura en el plano UV

4. Naturaleza de la RFI

4.1. Definicion y Origenes

La interferencia de radiofrecuencia (RFI) es cualquier senal electromagnetica no astronomica que contamina las observaciones. Los origenes principales incluyen:

1. **Comunicaciones terrestres:** Satelites, telefonia movil, WiFi
2. **Radar:** Militar, meteorologico, aeronautico
3. **Dispositivos electronicos:** Motores, equipos industriales
4. **Emission solar:** Actividad solar intensa

4.2. Caracteristicas de la RFI

La RFI se caracteriza por:

- Intensidad variable en tiempo y frecuencia
- Patrones espectrales especificos
- Correlacion entre antenas
- Estacionalidad y variabilidad diurna

4.3. Ejercicio de Investigacion 2

Objetivo: Analizar el impacto de RFI en diferentes tipos de observaciones.

Tareas:

1. Investigar las bandas de frecuencia mas afectadas por RFI
2. Comparar la sensibilidad de diferentes configuraciones de array
3. Analizar casos historicos de perdida de datos por RFI

5. Tipos de Interferencia

5.1. RFI de Banda Angosta

Caracteristicas:

- Ancho de banda < 1 MHz
- Frecuencia central fija
- Ejemplos: comunicaciones por satellite, radar

Impacto en visibilidades:

$$V_{corrupted}(u, v) = V_{true}(u, v) + A_{RFI}e^{i\phi_{RFI}} \quad (14)$$

donde:

- $V_{corrupted}(u, v)$: visibilidad contaminada por RFI
- $V_{true}(u, v)$: visibilidad verdadera (sin RFI)
- A_{RFI} : amplitud de la RFI (Jy)
- ϕ_{RFI} : fase aleatoria de la RFI (radianes)

5.2. RFI de Banda Ancha

Características:

- Ancho de banda > 10 MHz
- Puede cubrir multiples canales
- Ejemplos: WiFi, Bluetooth, equipos industriales

5.3. RFI Transitoria

Características:

- Duracion < 1 segundo
- Intensidad muy alta
- Ejemplos: chispas electricas, rayos

5.4. Ejercicio de Programacion 2

Objetivo: Simular fuentes puntuales usando cobertura UVW real.

Fórmulas necesarias:

1. Visibilidad para fuente puntual:

$$V(u, v) = S \cdot e^{-2\pi i(ul+vm)} \quad (15)$$

donde:

- $V(u, v)$: visibilidad compleja en coordenadas (u, v)
- S : flujo de la fuente puntual (Jy)
- u, v : coordenadas en el plano UV (longitudes de onda)
- l, m : cosenos directores de la fuente (radianes)

2. Visibilidad para múltiples fuentes:

$$V(u, v) = \sum_{k=1}^N S_k \cdot e^{-2\pi i(ul_k+vm_k)} \quad (16)$$

donde:

- N : numero de fuentes puntuales

- S_k : flujo de la fuente k (Jy)
- l_k, m_k : cosenos directores de la fuente k (radianes)

3. Ruido térmico:

$$\sigma_{thermal} = \frac{2k_B T_{sys}}{\sqrt{\Delta\nu\tau}} \quad (17)$$

donde:

- $\sigma_{thermal}$: desviacion estandar del ruido termico (Jy)
- k_B : constante de Boltzmann ($1,38 \times 10^{-23}$ J/K)
- T_{sys} : temperatura del sistema del telescopio (K)
- $\Delta\nu$: ancho de banda de observacion (Hz)
- τ : tiempo de integracion (s)

4. Relación señal-ruido:

$$SNR = \frac{S}{\sigma_{thermal}} \quad (18)$$

donde:

- SNR : relacion senal-ruido (adimensional)
- S : flujo de la fuente (Jy)
- $\sigma_{thermal}$: desviacion estandar del ruido termico (Jy)

Tareas:

1. Implementar la ecuación de visibilidad para fuentes puntuales
2. Simular múltiples fuentes con diferentes posiciones y flujos
3. Añadir ruido térmico realista
4. Calcular y visualizar SNR vs distancia UV
5. Analizar la distribución de amplitudes y fases

6. Tecnicas de Flagging

6.1. AOFlagger

AOFlagger es una herramienta automatica que utiliza multiples algoritmos:

1. **SumThreshold**: Detecta outliers en sumas de canales consecutivos
2. **Sliding window**: Analisis de ventanas deslizantes
3. **Statistical thresholding**: Umbrales basados en estadisticas

6.2. Algoritmo SumThreshold

Para cada ventana de tamaño w , calcula:

$$S_w = \sum_{i=1}^w |V_i| \quad (19)$$

Si $S_w > \theta_w$, marca los canales como RFI.

6.3. Umbrales Estadísticos

Utiliza la mediana y desviación mediana absoluta (MAD):

$$\text{threshold} = \text{median} + k \cdot \text{MAD} \quad (20)$$

donde k es un factor de escala típicamente entre 3-5.

6.4. Ejercicio de Programación 3

Objetivo: Generar ruido RFI sobre visibilidades reales.

Algoritmos necesarios:

1. RFI de banda angosta:

$$V_{RFI}(u, v) = A_{RFI} \cdot e^{i\phi_{RFI}} \cdot \delta(f - f_{RFI}) \quad (21)$$

donde:

- $V_{RFI}(u, v)$: visibilidad contaminada por RFI
- A_{RFI} : amplitud de la RFI (Jy)
- ϕ_{RFI} : fase aleatoria de la RFI (radianes)
- $\delta(f - f_{RFI})$: función delta de Dirac en frecuencia
- f_{RFI} : frecuencia central de la RFI (Hz)

2. RFI de banda ancha:

$$V_{RFI}(u, v) = A_{RFI} \cdot e^{i\phi_{RFI}} \cdot \Pi(f_{start}, f_{end}) \quad (22)$$

donde:

- A_{RFI} : amplitud de la RFI (Jy)
- ϕ_{RFI} : fase de la RFI (radianes)
- $\Pi(f_{start}, f_{end})$: función ventana rectangular
- f_{start}, f_{end} : frecuencias de inicio y fin de la RFI (Hz)

3. RFI transitoria:

$$V_{RFI}(t) = A_{RFI} \cdot e^{i\phi_{RFI}} \cdot \delta(t - t_{RFI}) \quad (23)$$

donde:

- $V_{RFI}(t)$: visibilidad contaminada en tiempo t
- t_{RFI} : tiempo de ocurrencia de la RFI (s)
- $\delta(t - t_{RFI})$: funcion delta de Dirac en tiempo

4. RFI correlacionada:

$$V_{RFI}(u, v) = A_{RFI} \cdot e^{i\phi_{RFI}} \cdot C(u, v) \quad (24)$$

donde:

- $C(u, v)$: funcion de correlacion espacial
- Correlacion alta para baselines cercanos en el plano UV

Tareas:

1. Implementar diferentes tipos de RFI según las fórmulas
2. Aplicar RFI a las visibilidades simuladas
3. Analizar el impacto en amplitud y fase
4. Visualizar la distribución de RFI en el plano UV
5. Calcular estadísticas de contaminación

7. Repaso Matematico

7.1. Transformada de Fourier

La transformada de Fourier discreta (DFT) se define como:

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-2\pi i k n / N} \quad (25)$$

donde:

- $X[k]$: coeficiente de Fourier en frecuencia k
- $x[n]$: muestra temporal en instante n
- N : numero total de muestras

Para radio-interferometria, la relacion entre visibilidades y distribucion de brillo es:

$$V(u, v) = \mathcal{F}\{I(l, m)\} \quad (26)$$

donde $\mathcal{F}$ representa la transformada de Fourier.

7.1.1. Matrices de Transformacion

En forma matricial, la transformada de Fourier se expresa como:

$$\mathbf{V} = \mathbf{A}\mathbf{I} \quad (27)$$

donde:

- $\mathbf{V}$: vector de visibilidades observadas
- $\mathbf{A}$: matriz de transformacion de Fourier
- $\mathbf{I}$: vector de imagen (pixeles)

7.1.2. Matriz Transpuesta y Adjunta

Matriz transpuesta $\mathbf{A}^T$:

- Se obtiene intercambiando filas y columnas
- $(A^T)_{ij} = A_{ji}$
- Para matrices reales: $\mathbf{A}^T$

Matriz adjunta $\mathbf{A}^H$ (hermitiana transpuesta):

- Se obtiene transponiendo Y conjugando elementos complejos
- $(A^H)_{ij} = A_{ji}^*$ (donde $*$ indica conjugado complejo)
- Para matrices complejas: $\mathbf{A}^H = (\mathbf{A}^*)^T$

Ejemplo: Si $\mathbf{A} = \begin{bmatrix} 1+i & 2-i \\ 3+2i & 4-i \end{bmatrix}$

Entonces $\mathbf{A}^H = \begin{bmatrix} 1-i & 3-2i \\ 2+i & 4+i \end{bmatrix}$

Importancia en radio-interferometria:

- $\mathbf{A}$: transforma de imagen a visibilidades
- $\mathbf{A}^H$: transforma de visibilidades a imagen (operacion inversa)
- Se usa en algoritmos de reconstruccion como gradiente conjugado

7.2. Espacios de Visibilidades

El espacio de visibilidades es el plano de Fourier del cielo. Propiedades importantes:

1. **Simetria:** $V(u, v) = V^*(-u, -v)$
 - Conjugacion compleja para coordenadas opuestas
 - Reduce el numero de visibilidades independientes
2. **Unitaridad:** Conservacion de energia

- La energia total se conserva en la transformacion
- Parseval: $\sum |V|^2 = \sum |I|^2$

3. Linealidad: Superposicion de fuentes

- $V_{total} = V_1 + V_2 + \dots + V_N$
- Permite analizar fuentes individualmente

7.3. Algebra Lineal y No Lineal

7.3.1. Problemas Lineales

Para problemas de calibracion y reconstruccion de imagenes basica:

$$\mathbf{V} = \mathbf{A}\mathbf{I} + \mathbf{n} \quad (28)$$

donde:

- $\mathbf{V}$: vector de visibilidades observadas
- $\mathbf{A}$: matriz de transformacion (transformada de Fourier)
- $\mathbf{I}$: vector de imagen (pixeles)
- $\mathbf{n}$: vector de ruido

7.3.2. Problemas No Lineales en Sintesis de Imagenes

La sintesis de imagenes en radio-interferometria es inherentemente no lineal debido a:

1. **Muestreo incompleto del plano uv**: La matriz $\mathbf{A}$ (transformada de Fourier) esta incompleta, lo que hace el problema mal condicionado
2. **Calibracion de fase**: Los errores de fase introducen no linealidad adicional
3. **Deconvolucion**: Algoritmos como CLEAN son iterativos y no lineales
4. **Modelado de fuentes**: Fuentes extendidas con estructura compleja

7.3.3. Ecuacion No Lineal General

Para sintesis de imagenes con muestreo incompleto:

$$\mathbf{V}_{obs} = \mathbf{A}_{incomplete}\mathbf{I} + \mathbf{n} \quad (29)$$

donde $\mathbf{A}_{incomplete}$ es la matriz de transformacion con muestreo incompleto del plano uv, haciendo necesario el uso de algoritmos no lineales como gradiente conjugado no lineal para la reconstruccion.

7.3.4. Algoritmos No Lineales

1. **CLEAN**: Deconvolucion iterativa
2. **Gradiente Conjugado No Lineal**: Para reconstruccion con muestreo incompleto
 - Mas eficiente que SVD para problemas grandes
 - Convergencia rapida en pocas iteraciones
 - Maneja bien el muestreo incompleto del plano UV
3. **Maximum Entropy**: Optimizacion con regularizacion
4. **Compressed Sensing**: Reconstruccion sparse

7.3.5. Ejercicio de Programacion 4

Objetivo: Implementar algoritmos basicos de flagging.

Algoritmos necesarios:

1. SumThreshold:

$$S_w = \sum_{i=1}^w |V_i| \quad (30)$$

Si $S_w > \theta_w$, donde $\theta_w = \text{median}(S_w) + k \cdot \text{MAD}(S_w)$.
donde:

- S_w : suma de amplitudes en ventana de tamaño w
- V_i : visibilidad compleja en posición i
- θ_w : umbral para ventana de tamaño w
- k : factor de escala (típicamente 3-5)
- MAD: desviación mediana absoluta

2. Umbral estadístico:

$$\text{threshold} = \text{median}(|V|) + k \cdot \text{MAD}(|V|) \quad (31)$$

donde:

- threshold: umbral estadístico global
- $|V|$: amplitudes de todas las visibilidades
- k : factor de escala (típicamente 3-5)

3. Flagging por variación de amplitud:

$$\text{threshold}_{bin} = \text{median}_{bin}(|V|) + k \cdot \text{MAD}_{bin}(|V|) \quad (32)$$

donde:

- threshold_{bin} : umbral para un bin específico

- median_{bin} : mediana dentro del bin
- MAD_{bin} : MAD dentro del bin
- Bin: grupo de visibilidades con distancias UV similares

4. Flagging por coherencia de fase:

$$\text{coherence} = \left| \frac{1}{N} \sum_{i=1}^N e^{i\phi_i} \right| \quad (33)$$

Si $\text{coherence} < \theta_{coh}$, marcar como RFI.
donde:

- coherence : medida de coherencia de fase (0-1)
- N : numero de visibilidades en el bin
- ϕ_i : fase de la visibilidad i (radianes)
- θ_{coh} : umbral de coherencia (tipicamente 0.3)

5. Metricas de evaluacion:

$$\text{Precision} = \frac{TP}{TP + FP} \quad (34)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (35)$$

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (36)$$

donde:

- TP : verdaderos positivos (RFI detectada correctamente)
- FP : falsos positivos (datos limpios marcados como RFI)
- FN : falsos negativos (RFI no detectada)
- TN : verdaderos negativos (datos limpios correctamente identificados)

Tareas:

1. Implementar algoritmo SumThreshold adaptado para coordenadas UVW
2. Implementar flagging por umbral estadistico
3. Implementar flagging por variacion de amplitud
4. Implementar flagging por coherencia de fase
5. Combinar diferentes algoritmos de flagging
6. Evaluar rendimiento con metricas de precision, recall y F1
7. Visualizar resultados de flagging en el plano UV

7.3.6. Ejercicio de Programacion 4.5

Objetivo: Implementar algoritmo CLEAN basico.

Algoritmo CLEAN:

1. Inicializacion:

$$\mathbf{I}_{clean} = \mathbf{0} \quad (37)$$

$$\mathbf{R} = \mathbf{I}_{dirty} \quad (38)$$

donde:

- $\mathbf{I}_{clean}$: modelo de imagen limpia (inicialmente cero)
- $\mathbf{R}$: imagen residual (inicialmente igual a imagen sucia)
- $\mathbf{I}_{dirty}$: imagen sucia inicial

2. Iteracion principal:

1. Encontrar pico maximo: $\mathbf{p} = \arg \max |\mathbf{R}|$
2. Si $|\mathbf{R}(\mathbf{p})| < \theta \cdot \max |\mathbf{I}_{dirty}|$, parar
3. Actualizar modelo: $\mathbf{I}_{clean}(\mathbf{p})+ = g \cdot \mathbf{R}(\mathbf{p})$
4. Actualizar residual: $\mathbf{R}(\mathbf{p})- = g \cdot \mathbf{R}(\mathbf{p})$

donde:

- $\mathbf{p}$: posicion del pico maximo en la imagen residual
- θ : umbral de parada (tipicamente 0.01-0.1)
- g : factor de limpieza (tipicamente 0.1-0.3)

3. Convolucion final:

$$\mathbf{I}_{final} = \mathbf{I}_{clean} \otimes \mathbf{B}_{clean} + \mathbf{R}_{final} \quad (39)$$

donde:

- $\mathbf{I}_{final}$: imagen final reconstruida
- $\otimes$: operacion de convolucion
- $\mathbf{B}_{clean}$: haz limpio (tipicamente Gaussiano)
- $\mathbf{R}_{final}$: residual final

Tareas:

1. Implementar algoritmo CLEAN iterativo
2. Generar imagen sucia con fuentes puntuales
3. Aplicar CLEAN con diferentes parametros (gain, threshold)
4. Visualizar modelo CLEAN y residual final
5. Comparar con imagen verdadera

7.4. Minimos Cuadrados

La solucion de minimos cuadrados para el sistema $\mathbf{Ax} = \mathbf{b}$ es:

$$\mathbf{x} = (\mathbf{A}^H \mathbf{A})^{-1} \mathbf{A}^H \mathbf{b} \quad (40)$$

7.5. Ejercicio de Programacion 5

Objetivo: Implementar reconstruccion de imagen usando gradiente conjugado no lineal.

Algoritmos necesarios:

1. Matriz de transformacion:

$$A_{ij} = e^{-2\pi i(u_i l_j + v_i m_j)} \quad (41)$$

donde:

- A_{ij} : elemento de la matriz de transformacion
- u_i, v_i : coordenadas UV de la visibilidad i
- l_j, m_j : coordenadas de pixel j en la imagen

2. Funcion objetivo (Chi-cuadrado):

$$\chi^2 = \sum_{i=1}^N \frac{|V_i^{obs} - V_i^{model}|^2}{\sigma_i^2} \quad (42)$$

donde:

- V_i^{obs} : visibilidad observada i
- V_i^{model} : visibilidad del modelo i
- σ_i : incertidumbre de la visibilidad i

3. Gradiente conjugado no lineal:

Input: $\mathbf{V}^{obs}$, $\mathbf{A}$, $\mathbf{I}_0$, tolerancia ϵ

Output: $\mathbf{I}_{final}$

Inicializacion::

$\mathbf{r}_0 = \mathbf{A}^H(\mathbf{V}^{obs} - \mathbf{A}\mathbf{I}_0)$;

$\mathbf{p}_0 = \mathbf{r}_0$;

$k = 0$;

Iteracion principal::

while $\|\mathbf{r}_k\| > \epsilon$ **do**

$\alpha_k = \frac{\mathbf{r}_k^H \mathbf{r}_k}{\mathbf{p}_k^H \mathbf{A}^H \mathbf{A} \mathbf{p}_k}$;

$\mathbf{I}_{k+1} = \mathbf{I}_k + \alpha_k \mathbf{p}_k$;

$\mathbf{r}_{k+1} = \mathbf{r}_k - \alpha_k \mathbf{A}^H \mathbf{A} \mathbf{p}_k$;

$\beta_k = \frac{\mathbf{r}_{k+1}^H \mathbf{r}_{k+1}}{\mathbf{r}_k^H \mathbf{r}_k}$;

$\mathbf{p}_{k+1} = \mathbf{r}_{k+1} + \beta_k \mathbf{p}_k$;

$k = k + 1$;

end

Resultado::

$\mathbf{I}_{final} = \mathbf{I}_k$;

Algorithm 1: Algoritmo de Gradiente Conjugado No Lineal

donde:

- $\mathbf{r}_k$: residual en iteracion k
- $\mathbf{p}_k$: direccion de busqueda en iteracion k
- α_k : paso optimo en iteracion k
- β_k : factor de conjugacion
- ϵ : tolerancia de convergencia

4. Regularizacion (opcional):

$$\chi_{reg}^2 = \chi^2 + \lambda \sum_{j=1}^M |I_j|^2 \quad (43)$$

donde:

- λ : parametro de regularizacion
- M : numero de pixeles en la imagen

Tareas:

1. Construir matriz de transformacion $\mathbf{A}$
2. Implementar algoritmo de gradiente conjugado no lineal
3. Calcular funcion objetivo chi-cuadrado
4. Aplicar regularizacion si es necesario
5. Reconstruir imagen y visualizar resultados
6. Comparar con imagen verdadera y evaluar convergencia

8. Ejercicios Integradores

8.1. Ejercicio de Investigacion 3

Título: Analisis de RFI en Frecuencias Bajas con Pyralysis

Objetivo: Analizar datos reales de radio-interferometria usando Pyralysis, enfocandose en RFI de frecuencias bajas.

Caracteristicas del RFI en frecuencias bajas:

- **Frecuencias afectadas:** < 1 GHz (bandas L, P, UHF)
- **Fuentes principales:**
 - Comunicaciones terrestres (FM, TV, celular)
 - Satelites de comunicacion
 - Radar meteorologico y militar
 - Interferencia domestica (WiFi, Bluetooth)
- **Patrones temporales:**
 - RFI persistente (24/7)
 - RFI diurna (horas comerciales)
 - RFI esporadica (eventos especiales)

Datos requeridos:

- Archivos MS (Measurement Set) de observaciones reales
- Datos de VLA, LOFAR, o MeerKAT preferiblemente
- Observaciones con RFI conocida en frecuencias bajas (< 1 GHz)
- **Nota:** ALMA opera en frecuencias altas (84-950 GHz) donde RFI es inexistente

Tareas con Pyralysis:

1. Cargar datos MS usando `pyralysis.io`
2. Inspeccionar estructura de datos y metadatos
3. Visualizar espectro de frecuencias para identificar RFI
4. Analizar patrones temporales de interferencia
5. Aplicar algoritmos de flagging implementados
6. Comparar imagenes antes y despues del flagging
7. Evaluar impacto del RFI en la calidad de imagen

Desarrollo colaborativo:

- **Repositorio principal:** <https://www.gitlab.com/clirai/pyralysis>

- **Proceso de desarrollo:**

1. Crear fork del repositorio principal
2. Clonar el fork localmente
3. Implementar cambios siguiendo lineamientos de programacion
4. Realizar testing exhaustivo
5. Crear pull request cuando todo este validado

- **Lineamientos de programacion:**

- Seguir PEP 8 para codigo Python
- Documentar funciones y clases
- Incluir tests unitarios y de integración
- Validar con datos reales

8.2. Ejercicio de Programacion 6

Título: Pipeline Completo de Deteccion de RFI con Pyralysis

Objetivo: Desarrollar un pipeline completo usando Pyralysis que combine multiples tecnicas de deteccion de RFI.

Enfoque en frecuencias bajas:

- **Analisis espectral:** Identificar picos de RFI en < 1 GHz
- **Correlacion temporal:** Detectar patrones diurnos/nocturnos
- **Correlacion espacial:** RFI que afecta multiples antenas
- **Thresholds adaptativos:** Ajustar segun frecuencia y hora

Pipeline con Pyralysis:

1. **Carga de datos:** Usar `pyralysis.io.MS()`
2. **Flagging automatico:**
 - SumThreshold para RFI persistente usando amplitud y fase
 - AOFlagger para RFI compleja con deteccion por coherencia
 - Thresholds estadisticos adaptativos por frecuencia
3. **Flagging manual:** Identificacion visual de RFI
4. **Generacion de imagen**
5. **Evaluacion de calidad:** Metricas de ruido y dinamica

Requisitos tecnicos:

1. Implementar pipeline usando Pyralysis desde fork personal
2. Anadir deteccion por correlacion entre antenas

3. Incluir analisis temporal de la RFI
4. Generar reportes de calidad de datos
5. Comparar diferentes estrategias de flagging
6. **Proceso de desarrollo:**
 - Crear fork de <https://www.gitlab.com/clirai/pyralysis>
 - Implementar cambios siguiendo lineamientos de programacion
 - Realizar testing exhaustivo con datos reales
 - Crear pull request cuando todo este validado

Analisis de frecuencias bajas:

- Identificar bandas mas afectadas por RFI
- Analizar correlacion entre RFI y hora del dia
- Evaluar efectividad de flagging por rango de frecuencia
- Comparar con modelos de RFI conocidos

9. Conclusiones y Proximos Pasos

Este taller proporciona los fundamentos necesarios para abordar la deteccion y eliminacion de RFI con tecnicas de aprendizaje profundo. Los ejercicios practicos permiten desarrollar habilidades en:

1. Analisis de datos interferometricos
2. Implementacion de algoritmos de flagging
3. Evaluacion de calidad de datos
4. Preparacion para tecnicas avanzadas de ML

9.1. Recursos Adicionales

Software principal:

- **Pyralysis:** <https://www.gitlab.com/clirai/pyralysis> - Framework principal para analisis de RFI
- **AOFlagger:** <https://sourceforge.net/p/aoflagger/> - Algoritmo de flagging automatico
- **CASA:** <https://casa.nrao.edu/> - Software de radio-interferometria

Desarrollo colaborativo:

- **Proceso de contribucion:**

1. Fork del repositorio Pyralysis
2. Desarrollo local con testing exhaustivo
3. Validacion con datos reales
4. Pull request siguiendo lineamientos

■ **Lineamientos de codigo:**

- PEP 8 para Python
- Documentacion completa
- Tests unitarios obligatorios
- Validacion con datos de observacion

Datos de observacion:

- **VLA Archive:** <https://archive.nrao.edu/> - Datos publicos del VLA (bandas L, C, X)
- **LOFAR Archive:** <https://lofar-surveys.org/> - Datos publicos de LOFAR (30-240 MHz)
- **MeerKAT Archive:** <https://archive.sarao.ac.za/> - Datos publicos de MeerKAT (0.58-14.5 GHz)
- **Nota:** ALMA (84-950 GHz) no presenta problemas de RFI

Rangos de frecuencia por observatorio:

- **VLA:**
 - Banda L: 1-2 GHz (RFI moderada)
 - Banda C: 4-8 GHz (RFI baja)
 - Banda X: 8-12 GHz (RFI muy baja)
- **LOFAR:** 30-240 MHz (RFI muy alta)
- **MeerKAT:** 0.58-14.5 GHz (RFI alta en frecuencias bajas)
- **ALMA:** 84-950 GHz (RFI inexistente)